Classification using Classical and Deep Learning Approaches

---

1. Introduction

The objective of this project is to develop an efficient and scalable classifier to predict the topic of a given text dataset containing approximately 10 million samples stored in a compressed Parquet format (~4GB).

Due to the large dataset size, special emphasis was placed on:

- Memory-efficient data handling
- Scalable model design
- Efficient training under hardware constraints

Both classical machine learning models and a custom deep learning model (CNN) were implemented from scratch, strictly adhering to the constraint of not using pretrained models.

---

2. Data Processing

2.1 Data Loading Strategy

Since the dataset is large, loading it entirely into memory was infeasible. Therefore:

- A subset of approximately 200K–300K samples was used
- Data was processed using efficient Parquet reading
- The design supports chunk-based processing for scalability

This ensured:

- Efficient memory usage
- Faster experimentation
- Scalability

2.2 Preprocessing

The following preprocessing steps were applied:

- Conversion to lowercase
- Removal of special characters
- Retention of alphanumeric tokens

Example:
"Hello World!!!" → "hello world"

2.3 Feature Engineering

1. Hashing Vectorizer

- Fixed dimensional representation (2^18 features)
- No vocabulary storage → memory efficient
- Configured with alternate_sign=False

Used for:

- Naive Bayes
- Logistic Regression
- SVM

---

2. TF-IDF (Optimized)

- Captures term importance
- Applied on a reduced subset (~50K samples)
- Feature size reduced (10K features)

---

3. Exploration & Iteration

3.1 Naive Bayes

- Fast baseline model
- Assumes feature independence

Observation:

- Performs reasonably well but limited in capturing complex patterns

---

3.2 Logistic Regression (SGD)

- Linear model
- Scales well with sparse data

Observation:

- Slight improvement over Naive Bayes

3.3 Support Vector Machine (SGD)

- Margin-based classifier

Observation:

- Strong performance on high-dimensional data

3.4 TF-IDF + Logistic Regression

- Standard NLP baseline

Observation:

- Performs well but computationally expensive for large data

3.5 CNN (Custom Model)

- Built from scratch using PyTorch
- Learns hierarchical representations

Observation:

- Best performing model
- Captures contextual relationships effectively

---

4. Final Model Architecture

The final model is a Convolutional Neural Network (CNN).

Architecture

Embedding Layer (Vocab ~50K, Dim = 128)
↓
Conv1D (kernel sizes: 3, 4, 5)
↓
ReLU Activation
↓
Global Max Pooling
↓
Concatenation
↓
Dropout (0.5)
↓
Fully Connected Layer
↓
Softmax Output

Parameter Count

- Embedding: ~6.4M
- Convolution layers: ~0.5M
- Fully connected: small

👉 Total ≈ 7–10 million parameters
✔️ Well below 5B constraint

---

5. Training Strategy

Data Split

- Training: 80%
- Validation: 20%

Hyperparameters

Parameter	Value
Batch Size	256
Epochs	2
Learning Rate	0.001
Optimizer	Adam

Hardware Used

- CPU (Intel i3)
- ~8GB RAM

Efficiency Considerations

- Subset-based training
- HashingVectorizer for scalability
- Optimized TF-IDF
- Reduced CNN epochs

---

6. Evaluation Metrics

- Accuracy
- Precision
- Recall
- F1 Score (weighted)

---

7. Final Results

Model	Accuracy	F1 Score
Naive Bayes	0.7999	0.7902

Model	Accuracy	F1 Score
Logistic Regression (Hashing)	0.8020	0.7923
SVM	0.8754	0.8707
TF-IDF + LR	0.8634	0.8556
CNN	0.8770	0.8729

---

8. Error Analysis

Observed Issues

- Confusion between similar topics
- Short text samples harder to classify
- Rare classes show lower performance

Patterns in Misclassification

- Overlapping vocabulary
- Lack of contextual depth

Potential Improvements

- Train CNN on larger dataset
- Introduce attention mechanism
- Apply advanced preprocessing (stopwords, stemming)
- Handle class imbalance

---

9. Conclusion

This project successfully demonstrates a scalable and efficient approach to large-scale text classification.

Key takeaways:

- Classical models provide strong baselines
- HashingVectorizer enables efficient large-scale processing
- CNN achieves the best performance by capturing contextual information

The final model meets all constraints and achieves strong performance under limited computational resources.